

Experiment 1

Fundamentals of Generative AI and Large Language Models

[Submission PDF](#)

[Source folder on GitHub](#)

Prepared 2026-09-29 from the experiment materials on branch
`codex/complete-experiments`.

Source: Ex01_GenAI_LLM_Fundamentals_Report/README.md

Ex-1: Comprehensive Report on the Fundamentals of Generative AI and LLMs

Due: 7 Aug 2026 **Reference repo:** RaajaThilahar/Ex.No.1

Objective

Compare the behaviour of different Generative AI models using **identical prompts**.

Tools compared

ChatGPT and Claude (primary pair). Gemini and Perplexity are optional extras - add outputs/gemini.md / outputs/perplexity.md if used.

Folder layout

Path	Purpose
prompts/	The 5 identical prompts (one file each)
outputs/	Saved model answers, one file per tool; completion times are marked not measured
comparison.md	Scored comparison on Accuracy, Creativity, Hallucination, Reasoning, Speed, Engineering usefulness
report.md	Final written report (fundamentals of GenAI + LLMs)

Procedure

The five prompts and corresponding saved answers are in prompts/ and outputs/.

The current comparison and report are complete; timing was not recorded and is identified as not measured.

Fresh timed runs can replace the saved answer records if the instructor requires them.

Source: Ex01_GenAI_LLM_Fundamentals_Report/LAB_RECORD.md

Lab Record: Ex-1: Comprehensive Report on Fundamentals of Generative AI and LLMs

Field	Entry
Name / Reg. No.	_____
Date	_____
Repo	https://github.com/RHUDHRESH/PROMPT_ENGINEERING-

Aim

Compare the behaviour of different generative AI models using identical prompts.

Tools / Apparatus

Claude (saved in outputs/model_b.md); ChatGPT (saved in outputs/chatgpt.md)

Procedure

Prompts written (see prompts/ or the folder README).

Each prompt run on every tool; raw outputs saved under outputs/ (or the folder's equivalent).

Results scored in comparison.md on: Accuracy, creativity, hallucination, reasoning, speed, engineering usefulness.

Findings summarised below.

Observations

See comparison.md. Add your own scores for any tool you ran yourself. Tools not run are left blank; do not fill them in from memory.

Result

Five prompt answers and an introductory report were prepared. The provisional desk review scores ChatGPT and Claude 19/25 each; response speed was not measured.

Conclusion

Identical prompts make it easier to compare model structure and coverage. The current answer set is useful for a first comparison, but timed fresh runs and independent human scoring are needed for a stronger result.

Faculty signature: _____

Source: [Ex01_GenAI_LLM_Fundamentals_Report/comparison.md](#)

Comparison: ChatGPT and Claude on identical prompts

Scores use 1 (poor) to 5 (strong). The ChatGPT response is from the current Codex session; the Claude responses were already present in the repository. These were not timed, freshly launched side-by-side runs. The quality scores are provisional desk reviews, not independent human ratings.

Criterion	What was checked	ChatGPT	Claude	Notes
Accuracy	Definitions, attention formula, scaling claims	4	4	Both give the standard attention equation and distinguish architecture families. The Claude answer's claims have not been independently checked; the ChatGPT report cites primary scaling papers but still needs faculty review.
Creativity	Analogies, examples, presentation	3	3	Both are structured and readable, with conventional examples rather than unusual analogies.
Hallucination control	Unsupported details and uncertainty	4	4	Both avoid precise undisclosed cost claims. The Claude response flags uncertainty. ChatGPT gives cautious general claims; paper titles and dates were checked against the linked primary papers in report.md.

Criterion	What was checked	ChatGPT	Claude	Notes
Reasoning	Explanations and trade-offs	4	4	Both explain architecture choices and scaling trade-offs.
Speed	Completion seconds	Not measured	Not measured	No stopwatch was used.
Engineering usefulness	Can a student use it to choose or build?	4	4	Both provide architecture comparisons and an end-to-end build outline; neither gives implementation code or cost estimates.
Total /25 scored	Speed omitted because it was not measured	19/25	19/25	Provisional desk-review scores; a tie here does not establish equal quality across tasks.

Hallucination and verification log

Tool	Claim	Check
Claude	Kaplan et al. (2020), <i>Scaling Laws for Neural Language Models</i>	Title and paper identifier are confirmed in report .md references.
Claude	Hoffmann et al. (2022), <i>Training Compute-Optimal Large Language Models</i>	Title and paper identifier are confirmed. The 20:1 token-to-parameter rule is an approximate summary, not a universal law.
Claude	Wei et al. (2022) emergent abilities paper	The repository output says it was unsure of the exact title. Do not cite that claim without checking the paper.
ChatGPT	Kaplan and Hoffmann scaling papers	Titles and identifiers were checked against arXiv records linked in report .md.
Both	Product-specific model architecture claims	Commercial implementation details may be undisclosed; treat examples as broad categories rather than verified internals.

Conclusion

Both answers cover the requested topics and offer useful introductory structure. The available records do not support a reliable speed ranking, and the Claude column was self-assessed in the original repository. Use the score table as a provisional exercise result; a fairer comparison would have a human reviewer score fresh answers from both tools without seeing their model labels.

Source: [Ex01_GenAI_LLM_Fundamentals_Report/report.md](#)

Report: Fundamentals of Generative AI and Large Language Models

1. Foundational concepts of Generative AI

Generative AI covers models that learn the distribution of their training data and can sample new, similar content: text, images, audio, code or video. This differs from discriminative models, which learn $p(y | x)$ and output a label or score, such as a spam classifier. A generative model instead learns $p(x)$, or $p(x, y)$, and can therefore create examples.

Four families are usually taught. Generative Adversarial Networks (GANs, Goodfellow et al., 2014) pit a generator against a discriminator. They produce sharp samples quickly but are hard to train and prone to mode collapse.

Variational Autoencoders (VAEs, Kingma and Welling, 2013) learn a smooth latent space through an encoder and decoder. They are stable but tend to produce blurrier output. Diffusion models (for example DDPM, Ho et al., 2020) generate by iteratively denoising random noise, giving high quality at the price of slow sampling. Autoregressive models factor the data distribution into next-element predictions, which underlies today's large language models. This split is a simplification, since real systems combine them (latent diffusion uses a VAE-style encoder).

Training data is mostly self-supervised: web text, code, image-caption pairs. Its quality, licensing and bias strongly shape behaviour. Common applications are code assistants, image generation for design, and synthetic data for domains where real data is scarce.

2. Generative AI architectures (Transformers)

The transformer (Vaswani et al., 2017) replaced recurrence with attention. Each token is projected to a query, key and value, and the output is

$\text{Attention}(Q, K, V) = \text{softmax}(Q K^T / d_k) V$.

The d_k scaling stops large dot products from saturating the softmax. Small example: with $d_k = 2$, query $[1, 0]$, keys $[1, 0]$ and $[0, 1]$, the scores are $[0.71, 0]$, the softmax weights are about $[0.67, 0.33]$, and the output is that weighted mix of the two value vectors. Multi-head attention runs several such operations in parallel so different heads can capture different relations. Because attention ignores order, positional encodings (sinusoidal in the original paper, often rotary today) are added.

Encoder-only models (BERT-style) suit understanding tasks, decoder-only models (GPT-style, with causal masking) suit generation, and encoder-decoder models suit translation and summarisation. Transformers displaced RNNs and LSTMs because they parallelise across the sequence, connect distant tokens in one step, and scale well with data and compute. The main cost is attention that grows quadratically with sequence length.

3. Architectures and applications

Each architecture has a natural niche. Transformers and LLMs are the most flexible tool for language, code and reasoning, with weaknesses in hallucination and cost. GANs give fast single-pass generation, useful for real-time image enhancement, but are unstable to train. VAEs suit representation learning, anomaly detection and latent compression. Diffusion models lead in image and increasingly video quality and are controllable through conditioning, though sampling is slow. Autoregressive audio and video models offer coherent long sequences for speech and music, at high generation cost; exact designs of several commercial systems are not public, so claims about them should be treated cautiously.

To choose, start from the modality and latency budget, then consider data and cost. Prefer adapting a pre-trained model (prompting, fine-tuning, retrieval) over training from scratch, and test on a small evaluation set that targets your worst failure mode.

4. Impact of scaling in LLMs

Loss on language modelling improves predictably, following power laws, as parameters, data and compute increase (Kaplan et al., 2020). Hoffmann et al. (2022, "Chinchilla") argued that for a fixed compute budget, parameters and training tokens should grow together, roughly 20 tokens per parameter, meaning many earlier models were undertrained. In practice, because inference cost recurs on every query, developers often train smaller models on much more data.

Some abilities appear to emerge suddenly at scale, though later analyses suggest part of this depends on how performance is measured, so the topic is contested. Returns diminish: each fixed gain in loss needs multiplicatively more compute, and high-quality text is finite. Training and serving are expensive and energy-hungry; precise figures are often undisclosed and should be treated as estimates. A newer axis is inference-time scaling, where extra computation at answer time (longer reasoning, multiple samples, verification) improves results on hard tasks.

5. What an LLM is and how it is built

An LLM is a very large neural network, usually a decoder-only transformer, trained to predict the next token. Construction proceeds in stages. Data is collected, deduplicated and filtered. Text is tokenized into subwords (for example byte-pair encoding). Pre-training on trillions of tokens builds general knowledge. Supervised fine-tuning on instruction-response examples teaches the model to follow instructions. Preference tuning, such as RLHF (Ouyang et al., 2022, InstructGPT) or simpler methods like DPO, aligns behaviour with human preferences. Evaluation uses benchmarks, human preference tests and red-teaming. Deployment relies on quantization, caching, batching and guardrails. Exact recipes differ between organisations and are often not disclosed, so this pipeline describes the typical case rather than any specific model.

6. Tool comparison summary (see [comparison.md](#))

The repository contains Claude answers saved earlier and ChatGPT answers produced in the current Codex session. Both were reviewed against the same five topics. The provisional desk-review scores are tied at 19/25 with speed omitted because it was not measured. The Claude scores were initially self-assessed by the model that supplied those answers; neither column is an independent human evaluation. Fresh, timed runs and blind human scoring would make the comparison stronger.

7. Conclusion

Generative AI has moved from specialised models such as GANs and VAEs to transformer-based LLMs and diffusion models, whose quality comes largely from scale and from careful post-training. Understanding the mechanics (attention, next-token prediction, fine-tuning, preference tuning) helps engineers pick the right architecture, set realistic expectations and design safeguards. The main risks remain hallucination, cost, bias and limited transparency. Comparing tools on identical prompts is useful, but conclusions should wait until every tool has actually been run and the claims verified.

References

- Vaswani, A., et al. (2017). [Attention Is All You Need](#). NeurIPS 2017.
- Kaplan, J., et al. (2020). [Scaling Laws for Neural Language Models](#).
- Hoffmann, J., et al. (2022). [Training Compute-Optimal Large Language Models](#) (Chinchilla).
- Ouyang, L., et al. (2022). [Training language models to follow instructions with human feedback](#) (InstructGPT). NeurIPS 2022.
- Goodfellow, I., et al. (2014). [Generative Adversarial Nets](#). NeurIPS 2014.
- Kingma, D. P., and Welling, M. (2013). [Auto-Encoding Variational Bayes](#).
- Ho, J., Jain, A., and Abbeel, P. (2020). [Denoising Diffusion Probabilistic Models](#). NeurIPS 2020.

Source: [Ex01_GenAI_LLM_Fundamentals_Report/outputs/chatgpt.md](#)

ChatGPT responses

Tool: ChatGPT in the current Codex session. Exact model version was not exposed. Date: 2026-09-29. Completion times were not measured. Each section answers the corresponding prompt in prompts/.

Q1 - Foundational concepts

Generative AI refers to models that learn patterns in examples and use them to create new content, such as text, images, audio, video or code. A generative model estimates a data distribution such as $p(x)$, or a conditional distribution such as $p(x|y)$, and samples from it. A discriminative model instead estimates a label or decision boundary, often $p(y|x)$.

Model family	How it generates	Example uses
Autoregressive	Predicts the next element repeatedly	Text, code, audio tokens
GAN	A generator competes with a discriminator	Image synthesis, augmentation
VAE	Encodes data into a latent distribution and decodes samples	Representation learning, controlled generation
Diffusion	Learns to remove noise over repeated steps	Images, audio and video

Training examples shape what a model can represent. Data filtering, licensing, coverage and bias affect the result; generated output is not guaranteed to be true or original. Applications include drafting and translation, image design, and engineering assistance such as code suggestions. The system still needs human review for factual, safety or rights-sensitive use.

Q2 - Generative AI architectures, focusing on transformers

A transformer splits input into tokens and maps each token to a vector. Self-attention lets each token combine information from other tokens. For queries Q , keys K and values V , scaled dot-product attention is $\text{Attention}(Q, K, V) = \text{softmax}(QK^T / d_k)V$. The scale d_k keeps dot products from growing too large as vector width increases. A query matches keys; the resulting weights blend the values. For example, weights 0.7 and 0.3 produce a context vector that is 70% of one value and 30% of another.

Multi-head attention runs several learned attention operations in parallel, allowing different heads to capture different relationships. Positional information is added because attention alone does not encode word order. Encoder-only models build contextual representations; decoder-only models predict the next token with a causal mask; encoder-decoder models encode an input and generate a conditioned output, useful for translation.

Transformers train efficiently on parallel hardware because they process a sequence together during training, unlike recurrent networks that step through tokens in order. Attention also creates shorter paths between distant tokens. The trade-off is attention cost that grows roughly quadratically with sequence length in the basic implementation; long-context models use optimisations to reduce memory or computation.

Q3 - Architectures and applications

Architecture	Strengths	Weaknesses	Example product or system	Engineering use
Transformer / LLM	Flexible language and code generation; instruction following	Can invent facts; high compute and context costs	ChatGPT, Gemini, Claude	Documentation drafts, code assistance, retrieval-based technical Q&A
GAN	Fast image sampling after training; sharp outputs	Training can be unstable; mode collapse	StyleGAN-family research systems	Synthetic inspection images for augmentation, with validation
VAE	Smooth latent space; useful probabilistic representation	Samples may look blurrier than GAN or diffusion outputs	VAE-based image and anomaly-detection systems	Compact sensor representation and anomaly exploration
Diffusion	High-quality, diverse image generation; controllable denoising	Iterative sampling can be compute-intensive	DALLE and Stable Diffusion systems	Concept imagery, synthetic visual data and design exploration
Autoregressive audio/video	Models temporal order and can condition on prior frames or tokens	Long sequences increase latency; temporal consistency remains difficult	Speech-generation and text-to-video systems	Voice interfaces, simulation assets and visualisation

Choose based on output type, quality and latency needs, available data and compute, and the cost of errors. Prototype with a baseline, measure task-specific quality, and add human review wherever mistakes could cause harm.

Q4 - Impact of scaling in LLMs

Scaling changes model parameters, training data and compute together. Kaplan et al. (2020) reported empirical power-law trends between language-model loss and scale. Hoffmann et al. (2022), in *Training Compute-Optimal Large Language Models*, showed that some large models were undertrained for their parameter count and argued for allocating more tokens per parameter at fixed compute. These findings guide planning; they do not guarantee a particular capability or product outcome.

More scale often improves average predictive performance, but each gain costs more data, accelerator time, energy and money. Diminishing returns appear when added compute yields smaller improvements on the target task. Claims of emergent abilities depend on the metric and evaluation scale; apparent jumps can arise when continuous improvements cross a threshold-based score.

Inference-time scaling spends more computation while answering, for example by sampling or checking multiple candidate solutions. It can improve selected reasoning tasks, but raises latency and cost and does not remove hallucinations. A practical choice balances measured task quality against training and serving cost, including smaller models, retrieval, caching and human checks.

Q5 - What an LLM is and how it is built

Stage	Purpose	Typical technique or tool
Data collection and filtering	Assemble relevant, permitted examples and remove poor-quality or sensitive records	Deduplication, policy filters, data documentation
Tokenization	Convert text into integer token IDs	Subword tokenizers such as BPE or SentencePiece
Pre-training	Learn broad language patterns by predicting tokens	Transformer training with next-token loss and distributed GPU/TPU jobs
Supervised fine-tuning	Teach the model to follow example instructions	Curated prompt-response examples and supervised optimization
Preference tuning	Align response choices with human preferences	Preference comparisons; RLHF or direct preference optimization
Evaluation	Measure capability, robustness, safety and regressions	Held-out benchmarks, red-team tests and human review
Deployment	Serve responses with limits, monitoring and updates	Model serving, rate limits, logging controls and evaluation gates

Remember: collect responsibly, turn text into tokens, train to predict, tune to follow instructions, test carefully, then serve with monitoring.

Source: [Ex01_GenAI_LLM_Fundamentals_Report/outputs/model_b.md](#)

Claude outputs

Model/version used: Claude (Sonnet 5.5, answered directly in a Claude Code session; not a fresh web chat) Date: 2026-09-29

Prompt	Time to complete (s)
Q1	not measured
Q2	not measured
Q3	not measured
Q4	not measured
Q5	not measured

Q1

Definition

Generative AI refers to models that learn the probability distribution of training data, $p(x)$, and can sample new data (text, images, audio, code) resembling it.

Discriminative vs generative

Aspect	Discriminative	Generative
Learns	$p(y \text{ given } x)$, a decision boundary	$p(x)$ or $p(x, y)$, the data distribution
Output	Label or number	New samples
Examples	Logistic regression, CNN classifier	GPT-style LLMs, GANs, diffusion
Typical use	Spam detection, diagnosis	Writing, image synthesis

Key model families

- **GANs:** a generator and a discriminator train adversarially. Samples are sharp, but training is unstable and can suffer mode collapse.
- **VAEs:** an encoder maps data to a latent distribution and a decoder reconstructs it. Training is stable and the latent space is smooth, but outputs tend to be blurrier.
- **Diffusion models:** learn to reverse a gradual noising process, generating by iterative denoising. Quality is high, but sampling is slow.
- **Autoregressive models:** factor $p(x)$ as a product of next-token probabilities. This is the basis of LLMs.

Training data

Models need large, diverse data: web text and code for LLMs, image-caption pairs for text-to-image systems. Data quality, duplication, licensing and bias directly shape model behaviour. Most training is self-supervised, so no manual labels are required.

Three applications

Code assistants that complete and explain code.

Text-to-image tools for design and prototyping.

Synthetic data generation, for example to augment scarce medical or driving datasets.

Uncertainty

The four-family split is a teaching simplification. Modern systems mix them (latent diffusion uses a VAE plus a transformer or U-Net), and diffusion can also be viewed as a hierarchical latent-variable model.

Q2

Overview

A transformer is a neural architecture built on attention rather than recurrence. It was introduced in "Attention Is All You Need" (Vaswani et al., 2017).

Self-attention (Q, K, V)

Each token embedding is projected into a query Q (what I am looking for), a key K (what I offer) and a value V (what I carry). Similarity between queries and keys decides how much of each value is mixed into the output.

$$\text{Attention}(Q, K, V) = \text{softmax}(Q K^T / d_k) V$$

Dividing by d_k keeps dot products from growing with dimension, which would push softmax into regions with tiny gradients.

Numeric intuition

Take $d_k = 2$, query $q = [1, 0]$, keys $k_1 = [1, 0]$ and $k_2 = [0, 1]$, values $v_1 = [10, 0]$ and $v_2 = [0, 10]$. Scores = $[1, 0] / 2 = [0.707, 0]$. Softmax gives weights of about $[0.67, 0.33]$. Output = $0.67v_1 + 0.33v_2$ $[6.7, 3.3]$. The token attends mostly, but not exclusively, to the more similar key.

Multi-head attention

Several attention operations run in parallel with different learned projections. Each head can specialise (for instance in syntax or coreference). Outputs are concatenated and linearly projected.

Positional encoding

Attention alone ignores order, so position information is added to the embeddings. The original paper used fixed sinusoids. Many modern models use learned or rotary (RoPE) encodings.

Encoder vs decoder vs encoder-decoder

- **Encoder-only** (BERT-style): bidirectional context, suited to classification and embeddings.
- **Decoder-only** (GPT-style): causal masking, so each token sees only earlier tokens. Suited to generation and now dominant for LLMs.
- **Encoder-decoder** (T5, original Transformer): the encoder reads the input and the decoder attends to it via cross-attention. Suited to translation and summarisation.

Why transformers replaced RNNs/LSTMs

RNNs process tokens sequentially, so they cannot be parallelised across a sequence. Transformers train much faster on GPUs.

Any two tokens are connected in one step, so long-range dependencies are easier to learn. In RNNs, signals must pass through many steps and gradients vanish.

They scale predictably with data and compute.

Trade-off: attention cost grows quadratically with sequence length, which motivates efficient-attention research.

Q3

Architecture	Strengths	Weaknesses	Example products	Typical engineering use case
Transformer / LLM	Flexible, strong language and reasoning, few-shot learning, scales well	Hallucination, high compute cost, quadratic attention	ChatGPT, Claude, Gemini, GitHub Copilot	Chatbots, code assistance, document Q&A with retrieval
GAN	Fast single-pass sampling, sharp images	Unstable training, mode collapse, weak coverage of the distribution	StyleGAN-based face generators, some super-resolution tools	Real-time image enhancement, style transfer, data augmentation
VAE	Stable training, smooth latent space, useful for compression	Blurrier outputs, posterior collapse	Often a component inside latent diffusion systems	Anomaly detection, representation learning, latent compression
Diffusion	Very high fidelity, diverse, controllable via conditioning	Slow iterative sampling, heavy compute	Stable Diffusion, DALL-E, Midjourney (architecture details for some are not public)	Image, video and design generation, inpainting
Autoregressive audio/video models	Coherent long sequences, conditioning on text	Slow token-by-token generation, high cost for video; some leading systems are proprietary, so exact architectures are uncertain	WaveNet-lineage speech models, text-to-speech and music tools, text-to-video systems (mix of diffusion and transformers)	Speech synthesis, music and sound design, video prototyping

Choosing an architecture: start from the output modality and the latency budget: text or code points to an LLM, images or video to diffusion, and real-time or on-device generation to a GAN or a distilled model. Then weigh data availability and cost, and prefer a pre-trained model you can prompt, fine-tune or ground with retrieval before training from scratch. Finally, prototype on a small evaluation set that measures the failure mode you fear most, such as hallucination, artifacts or latency, before committing.

Q4

Parameters, data, compute

Scaling means increasing model parameters (N), training tokens (D) and training compute (C). Loss falls smoothly and predictably as each grows.

Scaling laws

- **Kaplan et al., 2020, "Scaling Laws for Neural Language Models"** (I am confident this exists). It found power-law loss trends in N, D and C, and suggested that for a fixed budget, model size should grow faster than data.
- **Hoffmann et al., 2022, "Training Compute-Optimal Large Language Models"**, known as Chinchilla (confident it exists). It re-examined this and concluded that parameters and tokens should scale roughly equally, at around 20 tokens per parameter. Many earlier models were therefore undertrained. Treat "20:1" as an approximate rule of thumb.

Practice has since moved on: because inference cost is paid on every query, teams often train smaller models on far more tokens than Chinchilla-optimal.

Emergent abilities

Some skills, such as multi-step arithmetic or in-context learning, appear to show up abruptly beyond a certain scale. A paper by Wei et al. (2022) on emergent abilities exists, though I am not certain of its exact title. Later work argued that some apparent emergence is an artifact of discontinuous metrics. The debate is unresolved.

Diminishing returns

Power laws mean each constant improvement in loss needs multiplicatively more compute. Good-quality text data is also finite, so data availability may become a bottleneck, and lower loss does not automatically mean better real-world usefulness.

Cost and energy

Frontier training runs use thousands of accelerators for weeks, and are estimated to cost tens of millions of dollars or more. Exact figures and energy use are often undisclosed, so I would treat public estimates as uncertain. Inference at scale can exceed training in total energy.

Inference-time scaling

Spending more compute at answer time, through longer chain-of-thought, sampling several answers and voting, or search with verifiers, improves reasoning on hard tasks. It offers a second axis to trade cost against quality, though gains vary by task.

Q5

Stage	Purpose	Typical tool or technique
1. Data collection and cleaning	Gather a large, diverse and clean corpus so the model learns language and knowledge.	Web crawls, licensed and code datasets, deduplication (for example MinHash), quality and toxicity filters.
2. Tokenization	Convert text into integer tokens the model can process.	Subword methods such as byte-pair encoding (BPE) or SentencePiece.
3. Pre-training	Learn general language patterns by predicting the next token over trillions of tokens.	Decoder-only transformer, cross-entropy loss, AdamW, distributed training with PyTorch, DeepSpeed or Megatron-style parallelism on GPU/TPU clusters.
4. Supervised fine-tuning (SFT)	Teach the base model to follow instructions using example prompt-response pairs.	Curated human-written demonstrations, full fine-tuning or LoRA.
5. RLHF / preference tuning	Align outputs with human preferences for helpfulness and safety.	Reward model plus PPO (as in InstructGPT), or simpler methods like DPO; some labs also use AI feedback.
6. Evaluation	Measure capability, safety and regressions before release.	Benchmarks such as MMLU, human preference tests, red-teaming, task-specific test sets.
7. Deployment	Serve the model reliably, cheaply and safely to users.	Quantization, KV caching, batching in serving engines such as vLLM, plus guardrails, monitoring and retrieval or tool integration.

Exact recipes differ by lab and are often not public, so the techniques above are typical rather than universal.

5-line summary

- An LLM is a huge neural network trained to predict the next token.
- Clean data and tokenization decide what it can learn.
- Pre-training builds broad knowledge; fine-tuning teaches it to follow instructions.
- Preference tuning makes answers more helpful and safer, but not perfectly truthful.
- It still needs evaluation, guardrails and monitoring, because it can hallucinate.

Source: [Ex01_GenAI_LLM_Fundamentals_Report/prompts/q1_foundational_concepts.md](#)

Prompt 1 - Foundational concepts of Generative AI

You are a university AI lecturer. Explain the foundational concepts of Generative AI for a third-year engineering student.
Cover: definition, discriminative vs generative models, key model families (GANs, VAEs, diffusion, autoregressive), training data, and 3 real applications.
Limit to 400 words. Use headings and one comparison table. Flag anything you are unsure about.

Source: [Ex01_GenAI_LLM_Fundamentals_Report/prompts/q2_architectures_transformers.md](#)

Prompt 2 - Generative AI architectures (transformers)

Explain Generative AI architectures with a focus on transformers.
Cover: self-attention (Q, K, V), multi-head attention, positional encoding, encoder vs decoder vs encoder-decoder, and why transformers replaced RNNs/LSTMs.
Include the scaled dot-product attention formula and a small numeric intuition. Max 450 words.

Source: Ex01_GenAI_LLM_Fundamentals_Report/prompts/q3_architecture_applications.md

Prompt 3 - Architectures and their applications

Map each Generative AI architecture (Transformer/LLM, GAN, VAE, Diffusion, Autoregressive audio/video models) to its best-fit applications.

Return a table: Architecture | Strengths | Weaknesses | Example products | Typical engineering use case. Then give 3 sentences on how to choose an architecture for a new project.

Source: Ex01_GenAI_LLM_Fundamentals_Report/prompts/q4_scaling_impact.md

Prompt 4 - Impact of scaling in LLMs

Explain the impact of scaling on Large Language Models: parameters, data, and compute.

Cover scaling laws (Kaplan, Chinchilla), emergent abilities, diminishing returns, cost/energy, and inference-time scaling.

Cite the source paper names only if you are certain they exist; otherwise say "unsure". Max 400 words.

Source: Ex01_GenAI_LLM_Fundamentals_Report/prompts/q5_llm_how_built.md

Prompt 5 - What an LLM is and how it is built

Explain what a Large Language Model is and how it is built, end to end:

data collection & cleaning -> tokenization -> pre-training -> supervised fine-tuning -> RLHF/preference tuning -> evaluation -> deployment.

For each stage give one sentence on purpose and one on a typical tool or technique. Finish with a 5-line summary a beginner can remember.